

# Self-Driving Car

By Medhansh Mahesh Urs

## ABSTRACT:

This project demonstrates the development of a self-driving car system using a convolutional neural network (CNN) trained on Sully Chen's driving dataset. The model predicts steering angles based on images from a car's dashboard camera. The driving data was preprocessed, and the CNN architecture was optimized to achieve real-time steering angle predictions. The model's performance was evaluated on validation data, and results were visualized using real-time simulation.

## OBJECTIVE:

The primary objective of this project is to develop a self-driving car capable of predicting steering angles from front-facing camera images. This goal is achieved using supervised learning on Sully Chen's dataset, consisting of road images with labeled steering angles. The main goal of this project is to create a self-driving car that can predict steering angles using images from a front-facing camera. This will be done using supervised learning on Sully Chen's dataset, which contains road images with corresponding labeled steering angles. The network will be trained to imitate human driving behavior, emphasizing precise and smooth steering.

## INTRODUCTION:

Self-driving cars rely on a sophisticated system comprising sensors, cameras, and artificial intelligence algorithms to interpret their surroundings and autonomously navigate the road. The primary objective of our endeavor is to construct a self-driving model capable of accurately forecasting steering angles by leveraging a deep neural network. Our analysis is based on a dataset furnished by

Sully Chen, encompassing road images and their associated steering angles. The model processes these images by using a series of convolutional layers, ultimately resulting in the creation of an estimated steering angle.

## **METHODOLOGY:**

### **1. Dataset**

- a. We used Sully Chen's Dataset 1, which contains approximately 45,500 images from a dashboard-mounted camera, along with corresponding steering wheel angles. The steering wheel angles are converted from degrees to radians to ensure proper scaling for the network.

### **2. Data Preprocessing**

- a. To reduce any errors, I manually went through the 45,000+ lines of data and removed any outliers. There were 922 cases of the sensor registering a faulty reading. Almost 50% of the angles reported with  $0^\circ$  were inaccurate.
- b. The dataset was randomly shuffled and split into training (80%) and validation (20%) sets.
- c. Images were resized to 66x200 pixels and normalized to  $[0, 1]$  for faster convergence.

### **3. Model Architecture**

- a. The network structure is based on NVIDIA's complete self-driving vehicle model. It includes five convolutional layers for extracting features and fully connected layers to correlate the features with steering angles.
- b. Input: 66x200 pixel images.
- c. Convolutional Layers:
  - i. Conv1: 24 filters, 5x5 kernel @31x98.
  - ii. Conv2: 36 filters, 5x5 kernel @14x47.
  - iii. Conv3: 48 filters, 5x5 kernel @5x22.
  - iv. Conv4: 64 filters, 3x3 kernel @3x20.

- v. Conv5: 64 filters, 3x3 kernel @1x18.
    - vi.
  - d. Fully Connected Layers:
    - i. FC1: 1164 units.
    - ii. FC2: 100 units.
    - iii. FC3: 50 units.
    - iv. FC4: 10 units.
    - v. FC5: 1 output (steering angle).
  - e. Activation: ReLU was used after each convolution and fully connected layer.
  - f. Dropout: Applied to reduce overfitting.
  - g. Output: The predicted steering angle is scaled using the arctangent function for stability.

#### **4. Training**

- a. Loss Function: Mean Squared Error (MSE) between predicted and actual steering angles, with an added L2 regularization term to avoid overfitting.
- b. Optimizer: Adam optimizer with a learning rate of  $1e-4$  was used to minimize the loss function.
- c. The model was trained for 30 epochs with a batch size of 100, using TensorFlow's backend.
- d. The training process was monitored with TensorBoard for real-time tracking of the loss function.

#### **5. Validation**

- a. The validation set was used to evaluate model performance after every 10 batches. The model's ability to predict steering angles was analyzed through the validation loss, and predictions were visualized using a real-time simulation of the car's steering wheel.

## Code:

### 1) model.py

```
import tensorflow.compat.v1 as tf
tf.disable_v2_behavior()
import scipy

def weight_variable(shape):
    initial = tf.truncated_normal(shape, stddev=0.1)
    return tf.Variable(initial)

def bias_variable(shape):
    initial = tf.constant(0.1, shape=shape)
    return tf.Variable(initial)

def conv2d(x, W, stride):
    return tf.nn.conv2d(x, W, strides=[1, stride, stride, 1], padding='VALID')

x = tf.placeholder(tf.float32, shape=[None, 66, 200, 3])
y_ = tf.placeholder(tf.float32, shape=[None, 1])

x_image = x

W_conv1 = weight_variable([5, 5, 3, 24])
b_conv1 = bias_variable([24])

h_conv1 = tf.nn.relu(conv2d(x_image, W_conv1, 2) + b_conv1)

W_conv2 = weight_variable([5, 5, 24, 36])
b_conv2 = bias_variable([36])

h_conv2 = tf.nn.relu(conv2d(h_conv1, W_conv2, 2) + b_conv2)

W_conv3 = weight_variable([5, 5, 36, 48])
b_conv3 = bias_variable([48])

h_conv3 = tf.nn.relu(conv2d(h_conv2, W_conv3, 2) + b_conv3)

W_conv4 = weight_variable([3, 3, 48, 64])
b_conv4 = bias_variable([64])

h_conv4 = tf.nn.relu(conv2d(h_conv3, W_conv4, 1) + b_conv4)
W_conv5 = weight_variable([3, 3, 64, 64])
b_conv5 = bias_variable([64])

h_conv5 = tf.nn.relu(conv2d(h_conv4, W_conv5, 1) + b_conv5)

W_fc1 = weight_variable([1152, 1164])
b_fc1 = bias_variable([1164])

h_conv5_flat = tf.reshape(h_conv5, [-1, 1152])
h_fc1 = tf.nn.relu(tf.matmul(h_conv5_flat, W_fc1) + b_fc1)

keep_prob = tf.placeholder(tf.float32)
h_fc1_drop = tf.nn.dropout(h_fc1, keep_prob)

W_fc2 = weight_variable([1164, 100])
b_fc2 = bias_variable([100])

h_fc2 = tf.nn.relu(tf.matmul(h_fc1_drop, W_fc2) + b_fc2)

h_fc2_drop = tf.nn.dropout(h_fc2, keep_prob)

W_fc3 = weight_variable([100, 50])
b_fc3 = bias_variable([50])

h_fc3 = tf.nn.relu(tf.matmul(h_fc2_drop, W_fc3) + b_fc3)

h_fc3_drop = tf.nn.dropout(h_fc3, keep_prob)

W_fc4 = weight_variable([50, 10])
b_fc4 = bias_variable([10])

h_fc4 = tf.nn.relu(tf.matmul(h_fc3_drop, W_fc4) + b_fc4)

h_fc4_drop = tf.nn.dropout(h_fc4, keep_prob)

W_fc5 = weight_variable([10, 1])
b_fc5 = bias_variable([1])

y = tf.multiply(tf.atan(tf.matmul(h_fc4_drop, W_fc5) + b_fc5), 2)
```

## 2) driving\_data.py

```
import cv2
import random
import numpy as np

xs = []
ys = []

train_batch_pointer = 0
val_batch_pointer = 0

with open("driving_dataset/data.txt") as f:
    for line in f:
        xs.append("driving_dataset/" + line.split()[0])
        ys.append(float(line.split()[1]) * 3.14159265 / 180)

num_images = len(xs)

c = list(zip(xs, ys))
random.shuffle(c)
xs, ys = zip(*c)

train_xs = xs[:int(len(xs) * 0.8)]
train_ys = ys[:int(len(xs) * 0.8)]

val_xs = xs[-int(len(xs) * 0.2):]
val_ys = ys[-int(len(xs) * 0.2):]

num_train_images = len(train_xs)
num_val_images = len(val_xs)

def LoadTrainBatch(batch_size):
    global train_batch_pointer
    x_out = []
    y_out = []
    for i in range(0, batch_size):
        x_out.append(cv2.resize(cv2.imread(train_xs[(train_batch_pointer + i) % num_train_images])[-150:], (200, 66)) / 255.0)
        y_out.append([train_ys[(train_batch_pointer + i) % num_train_images]])
        train_batch_pointer += batch_size
    return x_out, y_out

def LoadValBatch(batch_size):
    global val_batch_pointer
    x_out = []
    y_out = []
    for i in range(0, batch_size):
        x_out.append(cv2.resize(cv2.imread(val_xs[(val_batch_pointer + i) % num_val_images])[-150:], (200, 66)) / 255.0)
        y_out.append([val_ys[(val_batch_pointer + i) % num_val_images]])
        val_batch_pointer += batch_size
    return x_out, y_out
```

### 3) train.py

```
import os
import tensorflow.compat.v1 as tf
tf.disable_v2_behavior()
from tensorflow.core.protobuf import saver_pb2
import driving_data
import model

LOGDIR = './save'

sess = tf.InteractiveSession()

L2NormConst = 0.001

train_vars = tf.trainable_variables()

loss = tf.reduce_mean(tf.square(tf.subtract(model.y_, model.y))) + tf.add_n([tf.nn.l2_loss(v) for v in train_vars]) * L2NormConst
train_step = tf.train.AdamOptimizer(1e-4).minimize(loss)
sess.run(tf.global_variables_initializer())

tf.summary.scalar("loss", loss)
merged_summary_op = tf.summary.merge_all()

saver = tf.train.Saver(write_version = saver_pb2.SaverDef.V2)

logs_path = './logs'
summary_writer = tf.summary.FileWriter(logs_path, graph=tf.get_default_graph())

epochs = 30
batch_size = 100

for epoch in range(epochs):
    for i in range(int(driving_data.num_images/batch_size)):
        xs, ys = driving_data.LoadTrainBatch(batch_size)
        train_step.run(feed_dict={model.x: xs, model.y_: ys, model.keep_prob: 0.8})
        if i % 10 == 0:
            xs, ys = driving_data.LoadValBatch(batch_size)
            loss_value = loss.eval(feed_dict={model.x:xs, model.y_: ys, model.keep_prob: 1.0})
            print("Epoch: %d, Step: %d, Loss: %g" % (epoch, epoch * batch_size + i, loss_value))

        summary = merged_summary_op.eval(feed_dict={model.x:xs, model.y_: ys, model.keep_prob: 1.0})
        summary_writer.add_summary(summary, epoch * driving_data.num_images/batch_size + i)
    if i % batch_size == 0:
        if not os.path.exists(LOGDIR):
            os.makedirs(LOGDIR)
        checkpoint_path = os.path.join(LOGDIR, "model.ckpt")
        filename = saver.save(sess, checkpoint_path)
        print("Model saved in file: %s" % filename)

print("Run the command line:\n" \
      "--> tensorboard --logdir=./logs " \
      "\nThen open http://0.0.0.0:6006/ into your web browser")
```

#### 4) run\_dataset.py

```
import tensorflow.compat.v1 as tf
tf.disable_v2_behavior()
import model
import cv2
from subprocess import call
import os

windows = False
if os.name == 'nt':
    windows = True

sess = tf.InteractiveSession()
saver = tf.train.Saver()
saver.restore(sess, "save/model.ckpt")

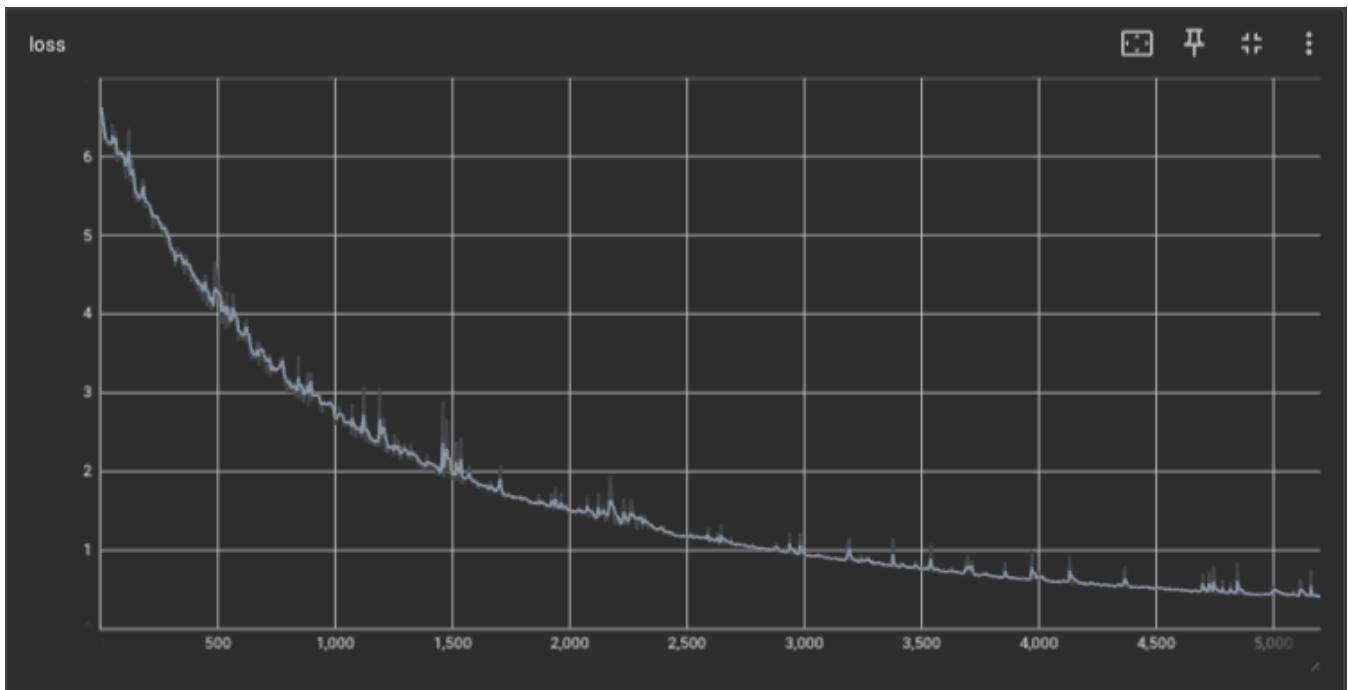
img = cv2.imread('steering_wheel_image.jpg',0)
rows,cols = img.shape

smoothed_angle = 0

i = 0
while(cv2.waitKey(10) != ord('q')):
    full_image = cv2.imread("driving_dataset/" + str(i) + ".jpg")
    image = cv2.resize(full_image[-150:], (200, 66)) / 255.0
    degrees = model.y.eval(feed_dict={model.x: [image], model.keep_prob: 1.0})[0][0] * 180.0 / 3.14159265
    if not windows:
        call("clear")
    print("Predicted steering angle: " + str(degrees) + " degrees")
    cv2.imshow("frame", full_image)
    smoothed_angle += 0.2 * pow(abs((degrees - smoothed_angle)), 2.0 / 3.0) * (degrees - smoothed_angle) / abs(degrees - smoothed_angle)
    M = cv2.getRotationMatrix2D((cols/2,rows/2),-smoothed_angle,1)
    dst = cv2.warpAffine(img,M,(cols,rows))
    cv2.imshow("steering wheel", dst)
    i += 1

cv2.destroyAllWindows()
```

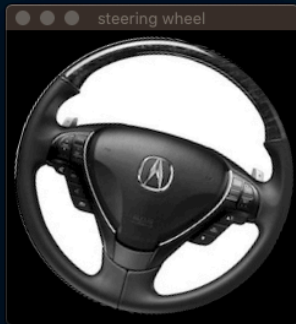
#### 5) Loss graph till Epoch: 10, Step: 1440, Loss: 0.462273





6) Driver taking slight right turn

Predicted steering angle: 17.4394042069578 degrees



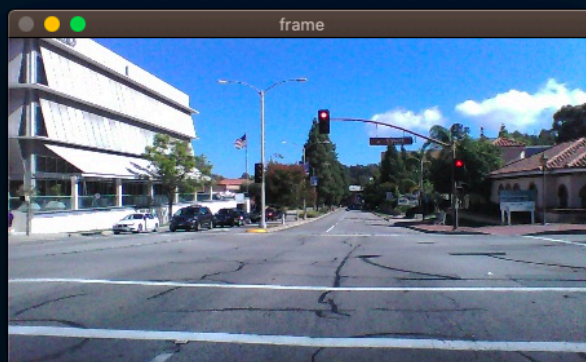
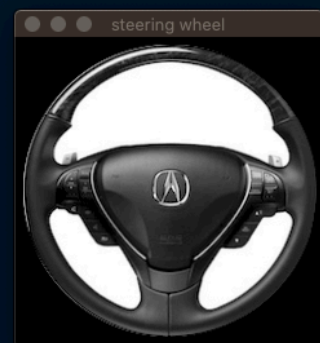
7) Driver taking slight left turn

Predicted steering angle: -2.8437883243286266 degrees



8) Driver waiting for straight signal

Predicted steering angle: 0.6122817967574076 degrees



## CONCLUSION:

In this project, we developed a deep learning model for self-driving cars that accurately predicts steering angles from camera images. Using a CNN architecture, we successfully trained the model on Sully Chen's dataset. The model performed well during validation, and the real-time simulation showcased its ability to predict smooth steering angles. Future work may involve refining the model, adding more data, and testing the system in real-world environments.